Name – Garima Uppal

Sec- B

Roll – 22

Cu240251618

Experiment No. 8

Experiment Name

Customer Churn Prediction using Decision Tree Classification

Aim

To build and evaluate a Decision Tree Classification model for predicting customer churn using a real-world dataset and analyze the factors influencing customer retention.

GITHUB LINK: https://github.com/suyash34501/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement/blob/main/Experiment%208.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
from google.colab import files

uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.
Saving customer churn decision tree dataset.csv to customer churn decision tree dataset.csv

```
import io

filename = list(uploaded.keys())[0]

df = pd.read_csv(io.BytesIO(uploaded[filename]))

print("Dataset Loaded Successfully!")
print("Shape:", df.shape)

df.head()
```

```
Dataset Loaded Successfully!
Shape: (250, 10)
```

	CustomerID	Gender	SeniorCitizen	TenureMonths	MonthlyCharges	Contract	InternetService	TechSupport	Payment
0	CUST0001	Male	0	8	28.02	One year	Fiber optic	Yes	Bank
1	CUST0002	Female	1	26	77.13	Month-to-month	Fiber optic	No	El
2	CUST0003	Male	1	34	75.77	Month-to-month	Fiber optic	No	Cre
3	CUST0004	Male	0	7	58.82	One year	DSL	No	El

```
print("Dataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
display(df.describe(include="all"))
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250 entries, 0 to 249
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       250 non-null    object
 1   Gender           250 non-null    object
 2   SeniorCitizen    250 non-null    int64
 3   TenureMonths     250 non-null    int64
 4   MonthlyCharges   250 non-null    float64
 5   Contract         250 non-null    object
 6   InternetService  250 non-null    object
 7   TechSupport      250 non-null    object
 8   PaymentMethod    250 non-null    object
 9   Churn            250 non-null    object
dtypes: float64(1), int64(2), object(7)
memory usage: 19.7+ KB

Missing Values:
CustomerID         0
Gender             0
SeniorCitizen      0
TenureMonths       0
MonthlyCharges     0
Contract           0
InternetService    0
TechSupport        0
PaymentMethod      0
Churn              0
dtype: int64

Statistical Summary:
```

	CustomerID	Gender	SeniorCitizen	TenureMonths	MonthlyCharges	Contract	InternetService	TechSupport	Pa
count	250	250	250.000000	250.000000	250.000000	250	250	250	
unique	250	2	NaN	NaN	NaN	3	3	2	
top	CUST0001	Male	NaN	NaN	NaN	Month-to-month	Fiber optic	No	
freq	1	126	NaN	NaN	NaN	128	122	170	
mean	NaN	NaN	0.220000	35.416000	72.315520	NaN	NaN	NaN	
std	NaN	NaN	0.415077	20.636142	26.704319	NaN	NaN	NaN	
min	NaN	NaN	0.000000	1.000000	25.250000	NaN	NaN	NaN	
25%	NaN	NaN	0.000000	17.000000	47.625000	NaN	NaN	NaN	
50%	NaN	NaN	0.000000	35.000000	74.635000	NaN	NaN	NaN	
75%	NaN	NaN	0.000000	53.750000	94.985000	NaN	NaN	NaN	

```
df = df.drop("CustomerID", axis=1)

# Check missing values
print("Missing values before preprocessing:")
print(df.isnull().sum())

# Handle missing values
df = df.dropna()

print("\nDataset after preprocessing:")
display(df.head())
```

```
Missing values before preprocessing:
Gender             0
SeniorCitizen      0
TenureMonths       0
MonthlyCharges     0
Contract           0
InternetService    0
TechSupport        0
PaymentMethod      0
Churn              0
dtype: int64

Dataset after preprocessing:
```

	Gender	SeniorCitizen	TenureMonths	MonthlyCharges	Contract	InternetService	TechSupport	PaymentMethod	Chur
0	Male	0	8	28.02	One year	Fiber optic	Yes	Bank transfer	N
1	Female	1	26	77.13	Month-to-month	Fiber optic	No	Electronic check	Ye
2	Male	1	34	75.77	Month-to-month	Fiber optic	No	Credit card	Ye
3	Male	0	7	58.82	One year	DSL	No	Electronic	N

```
label_encoder = LabelEncoder()

categorical_columns = [
    "Gender",
    "Contract",
    "InternetService",
    "TechSupport",
    "PaymentMethod",
    "Churn"
]

for column in categorical_columns:
    df[column] = label_encoder.fit_transform(df[column])

print("Categorical variables encoded successfully!")

display(df.head())
```

```
Categorical variables encoded successfully!
```

	Gender	SeniorCitizen	TenureMonths	MonthlyCharges	Contract	InternetService	TechSupport	PaymentMethod	Chur
0	1	0	8	28.02	1	1	1	0	
1	0	1	26	77.13	0	1	0	2	
2	1	1	34	75.77	0	1	0	1	
3	1	0	7	58.82	1	0	0	2	
4	1	0	68	109.95	0	1	0	2	

```
X = df.drop("Churn", axis=1)
y = df["Churn"]

print("Features:")
print(X.columns)

print("\nTarget Variable:")
print("Churn")
```

```
Features:
Index(['Gender', 'SeniorCitizen', 'TenureMonths', 'MonthlyCharges', 'Contract',
       'InternetService', 'TechSupport', 'PaymentMethod'],
      dtype='object')

Target Variable:
Churn
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (200, 8)
Testing Data: (50, 8)
```

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)

print("Decision Tree Model Trained Successfully!")
```

```
Decision Tree Model Trained Successfully!
```

```
y_pred = model.predict(X_test)

print("Predictions:")
print(y_pred)
```

```
Predictions:
[1 0 0 0 0 0 0 0 1 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 0 0 1 1 1 0 0 1 1 0 1 0
 1 1 1 1 0 1 0 0 0 0 0 0 0 0]
```

```
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test, y_pred, zero_division=0
)

recall = recall_score(
    y_test, y_pred, zero_division=0
)

f1 = f1_score(
    y_test, y_pred, zero_division=0
)

print("Model Evaluation Results")
print("------------------------")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
```

```
Model Evaluation Results
------------------------
Accuracy : 0.56
Precision: 0.5
Recall   : 0.4545
F1-Score : 0.4762
```

```
print("Classification Report:")
print(
```

```
    classification_report(
        y_test,
        y_pred,
        target_names=["No Churn", "Churn"],
        zero_division=0
    )
)
```

```
Classification Report:
              precision    recall  f1-score   support

    No Churn       0.60      0.64      0.62        28
       Churn       0.50      0.45      0.48        22

    accuracy                           0.56        50
   macro avg       0.55      0.55      0.55        50
weighted avg       0.56      0.56      0.56        50
```

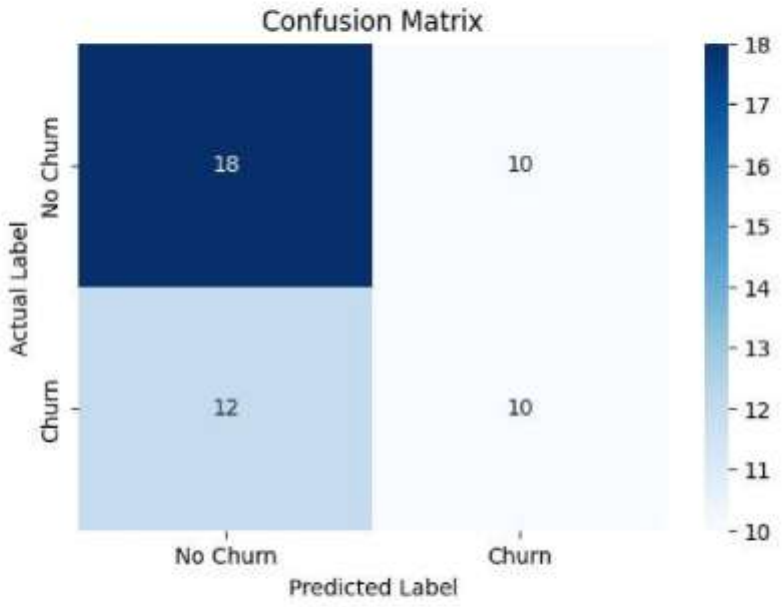
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")

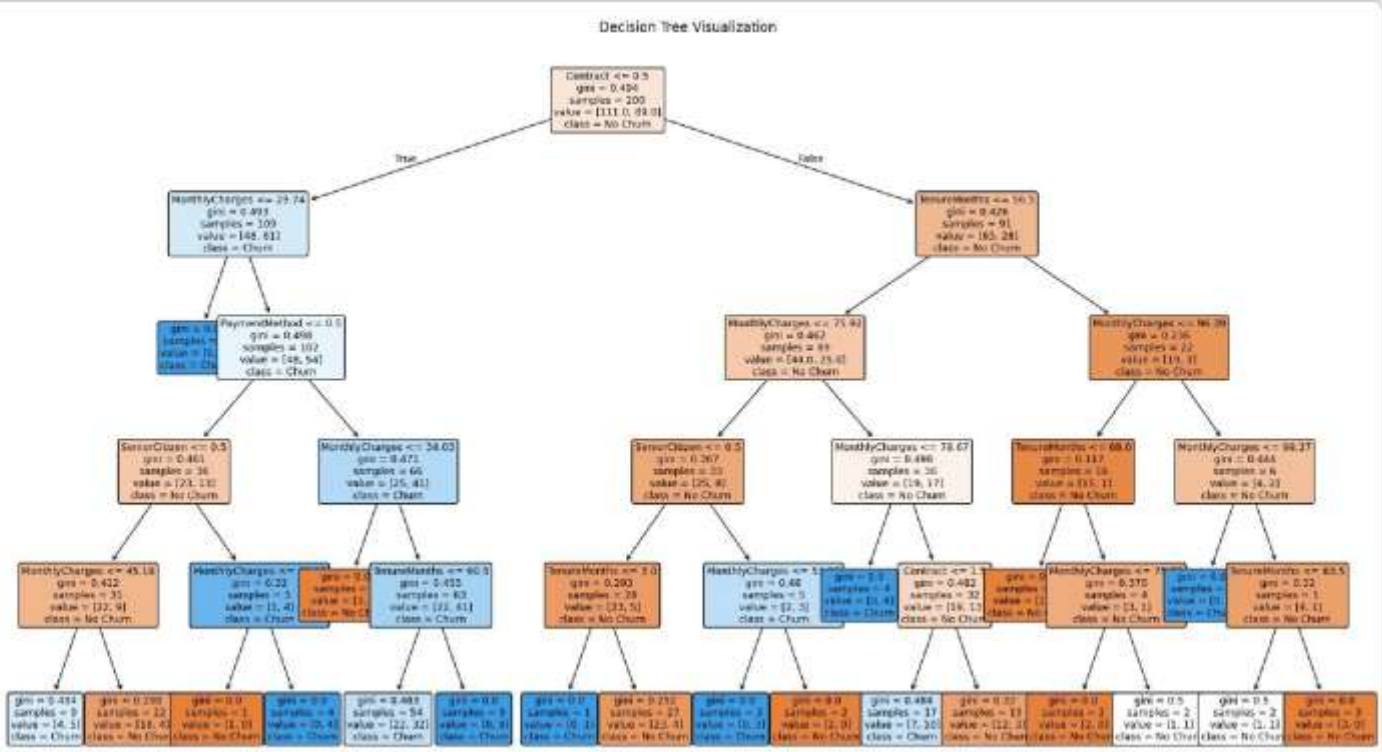
plt.show()
```




```
plt.figure(figsize=(22, 12))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    rounded=True,
    fontsize=9
)

plt.title("Decision Tree Visualization")
plt.show()
```




```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("Feature Importance:")
display(importance)
```

```
Feature Importance:
```

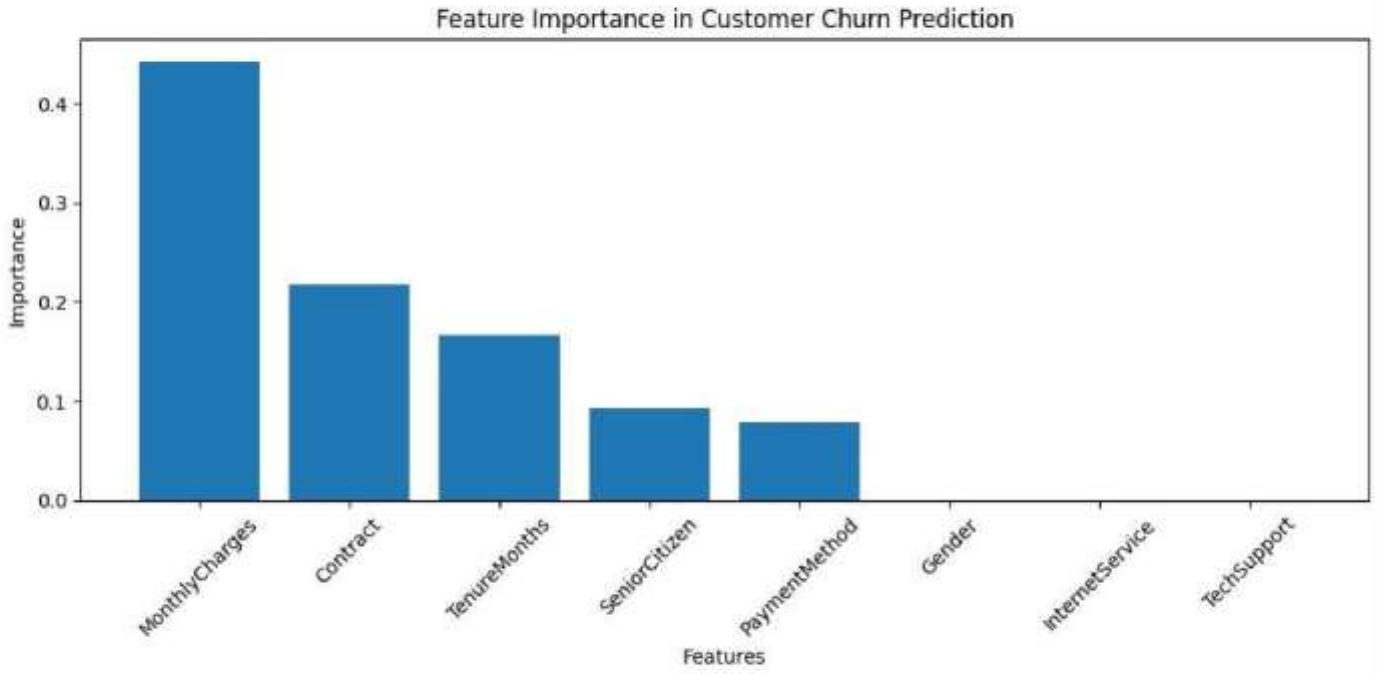
	Feature	Importance
3	MonthlyCharges	0.442457
4	Contract	0.218132
2	TenureMonths	0.166483
1	SeniorCitizen	0.093887
7	PaymentMethod	0.079040
0	Gender	0.000000
5	InternetService	0.000000
6	TechSupport	0.000000

```
plt.figure(figsize=(10, 5))
```

```
plt.bar(
    importance["Feature"],
    importance["Importance"]
)

plt.xticks(rotation=45)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance in Customer Churn Prediction")

plt.tight_layout()
plt.show()
```




```
new_customer = pd.DataFrame([{
    "Gender": 1,
    "SeniorCitizen": 0,
    "TenureMonths": 5,
    "MonthlyCharges": 95.0,
    "Contract": 0,
    "InternetService": 1,
    "TechSupport": 0,
    "PaymentMethod": 0
}])

prediction = model.predict(new_customer)

if prediction[0] == 1:
    print("Prediction: Customer is likely to churn.")
else:
    print("Prediction: Customer is likely to stay.")
```

```
Prediction: Customer is likely to stay.
```

Q1. What is classification, and how does it differ from regression?

Answer: Classification is a supervised machine learning technique used to predict categorical output classes, such as Yes or No. Regression predicts continuous numerical values, such as salary or house price.

Q2. Explain the working principle of the Decision Tree algorithm.

Answer: A Decision Tree divides data into smaller groups using feature-based conditions. It starts from a root node and creates branches based on splitting criteria. The process continues until leaf nodes produce predictions.

Q3. Differentiate between Gini Index and Entropy.

Answer: Gini Index measures impurity based on the probability of incorrect classification. Entropy measures the uncertainty or disorder in data using information theory. Both can be used to select suitable splits in a Decision Tree.

Q4. What is a Confusion Matrix?

Answer: A Confusion Matrix evaluates classification predictions using four components: True Positive, True Negative, False Positive, and False Negative. It helps identify correct and incorrect predictions.

Q5. Define Accuracy, Precision, Recall, and F1-Score.

Answer:

- **Accuracy:** Proportion of all predictions that are correct.
- **Precision:** Proportion of predicted positive cases that are actually positive.
- **Recall:** Proportion of actual positive cases correctly identified.
- **F1-Score:** Harmonic mean of precision and recall.

These metrics help evaluate different aspects of model performance.

Q6. What is overfitting in a Decision Tree?

Answer: Overfitting occurs when a Decision Tree learns training data too closely, including noise, and performs poorly on unseen data. It can be reduced by limiting tree depth, increasing minimum samples per leaf, and using pruning.

Q7. Why is customer churn prediction important?

Answer: Customer churn prediction helps organizations identify customers who may leave their services. Businesses can use these insights to develop retention strategies, improve customer satisfaction, and reduce potential revenue loss.

Q8. What are the advantages and limitations of Decision Trees?

Answer:

Advantages:

- Easy to understand and interpret.
- Can handle numerical and categorical features after suitable preprocessing.
- Requires relatively little data preparation.

Limitations:

- Can overfit training data.
- Small data changes can produce different trees.
- A single tree may have lower predictive performance than some ensemble methods.

Q9. Mention three real-world applications of Decision Tree Classification.

Answer:

1. Medical disease classification.
2. Loan approval and credit risk assessment.
3. Email spam detection.

Q10. How can churn prediction improve customer retention?

Answer: Organizations can identify customers who may leave and analyze the factors associated with churn. They can then offer suitable support, improve services, and develop targeted retention programs. This may help improve customer loyalty and business profitability.
